

Assignment 2: Dates Generative Models

Deep Learning — Conditional Date Generation using Neural Networks

1. Problem Overview

The task is to generate a valid date given four conditions. Each input line looks like this:

[WED] [JAN] [False] [180]

The model must output a real calendar date that matches all four conditions, for example:
3-1-1800

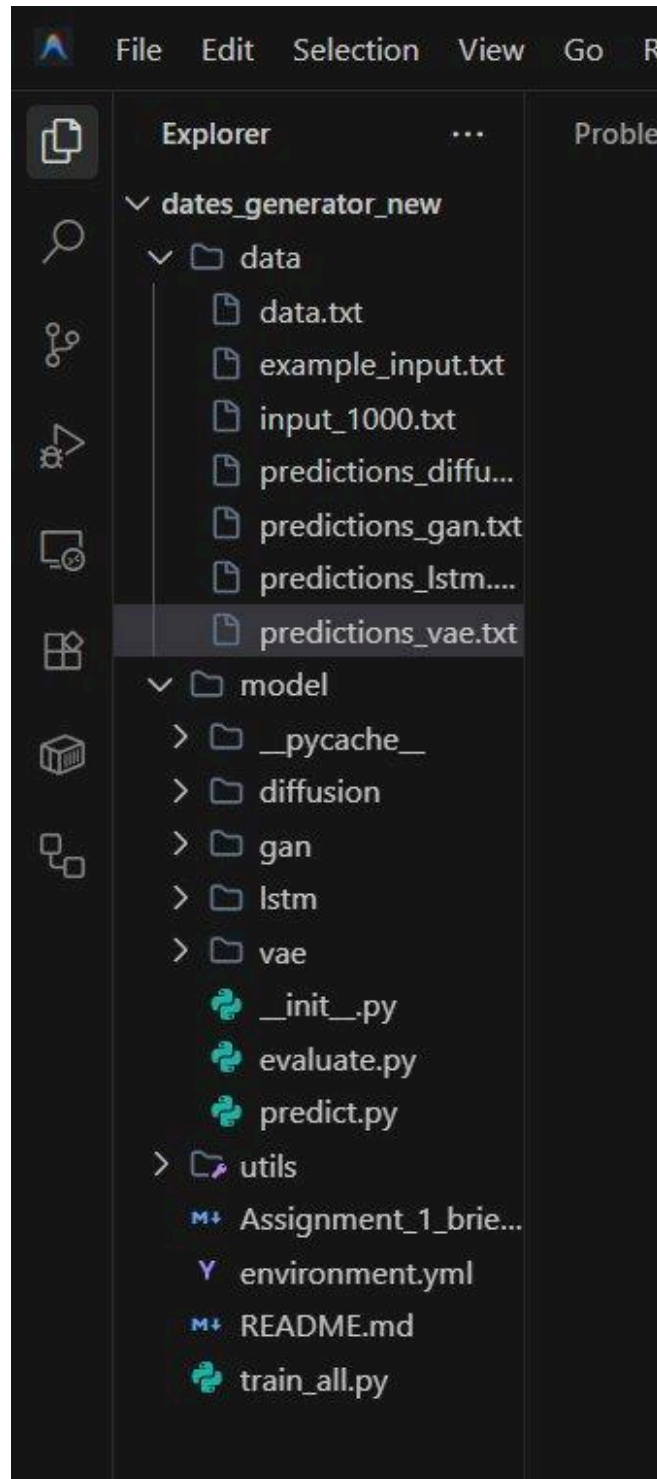
The four conditions are the day of the week, the month, whether the year is a leap year, and the decade. Many valid dates can satisfy the same conditions, so this is a generation problem, not a classification problem.

2. Dataset

The dataset contains 146,000 entries. Each line has the four condition tokens followed by a target date. 90% of lines are used for training and 10% for validation.

3. Project File Structure

```
dates_generator/
├── data/
│   ├── data.txt
│   ├── example_input.txt
│   └── input_1000.txt      1000-line evaluation set
├── model/
│   ├── predict.py         main inference entry point
│   ├── evaluate.py        scoring with per-condition breakdown
│   ├── gan/
│   │   └── gan_model.py   Model 1: Conditional GAN
│   ├── vae/
│   │   └── vae_model.py   Model 2: Conditional VAE
│   ├── lstm/
│   │   └── lstm_model.py  Model 3: LSTM Seq2Seq with Attention
│   └── diffusion/
│       └── diffusion_model.py Model 4: Discrete Denoising Diffusion
├── utils/
│   ├── tokenizer.py       shared vocabulary, encode/decode, validation
│   └── dataset.py         PyTorch Dataset wrappers
├── train_all.py           trains one or all models
└── environment.yml
```



4. Tokenization

Each condition token is converted to an integer ID using a fixed vocabulary. The vocabulary includes all 7 day tokens, 12 month tokens, 2 leap year tokens, 41 decade tokens, digit tokens 0

through 9, and special tokens for padding, start of sequence, and end of sequence. The total vocabulary size is 76.

The output date is not treated as a raw string. Instead it is broken into 8 individual digit tokens. For example 3-12-1962 becomes the sequence:

["0", "3", "1", "2", "1", "9", "6", "2"]

This makes the generation space much smaller: 8 positions each with only 10 possible values. Sequence models additionally use a start token before and an end token after these 8 digits, giving sequences of length 10.

5. Why Loss, Not Accuracy, Is Tracked During Training

Accuracy requires comparing a generated date against a correct answer. But there is no single correct answer: any date that satisfies all four conditions is valid. More importantly, accuracy is a step function that is not differentiable, so gradients cannot flow through it during backpropagation. Each model therefore minimizes a continuous surrogate loss during training. After training, constraint satisfaction rate is used to evaluate output quality offline.

6. Models

Four models were implemented. Two are from the course and two are from outside the course.

Model 1: Conditional GAN

The GAN has two networks. The Generator takes the four condition embeddings plus a random Gaussian noise vector and produces eight digit logits. The Discriminator takes the same condition embeddings plus a digit sequence and outputs a real or fake score. During training the Discriminator is trained to tell apart real dates from generated ones, and the Generator is trained to fool the Discriminator.

Architecture:

The Generator uses a condition embedder that projects the four tokens into a 256-dimensional vector, concatenates it with a projected noise vector, and passes it through two linear layers with ReLU activations and LayerNorm to produce 8x10 logits. The Discriminator uses the same condition embedder plus digit embeddings, concatenated and passed through two LeakyReLU layers to a scalar score.

Loss functions:

Discriminator loss = $\text{softplus}(-\text{real score}) + \text{softplus}(\text{fake score})$

Generator loss = $\text{softplus}(-\text{fake score})$

Training observations:

Both Loss_D and Loss_G stabilized around 1.38 and 0.70 respectively after epoch 50, which is normal GAN behavior. The losses do not converge to zero; a balanced adversarial equilibrium is the expected outcome.

Model 2: Conditional VAE

The VAE has an Encoder that takes the condition tokens and the real digit sequence and produces a mean vector and a log-variance vector in a 64-dimensional latent space. The Decoder takes the condition tokens and a sampled latent vector and produces 8 digit logits.

Architecture:

The Encoder embeds all 8 digit positions using a digit embedding, concatenates them with the condition embedding, and projects through two linear layers to produce the mean and log variance. The Decoder concatenates the condition embedding with a sampled z and passes it through two linear layers with LayerNorm and ReLU to produce 8×10 logits.

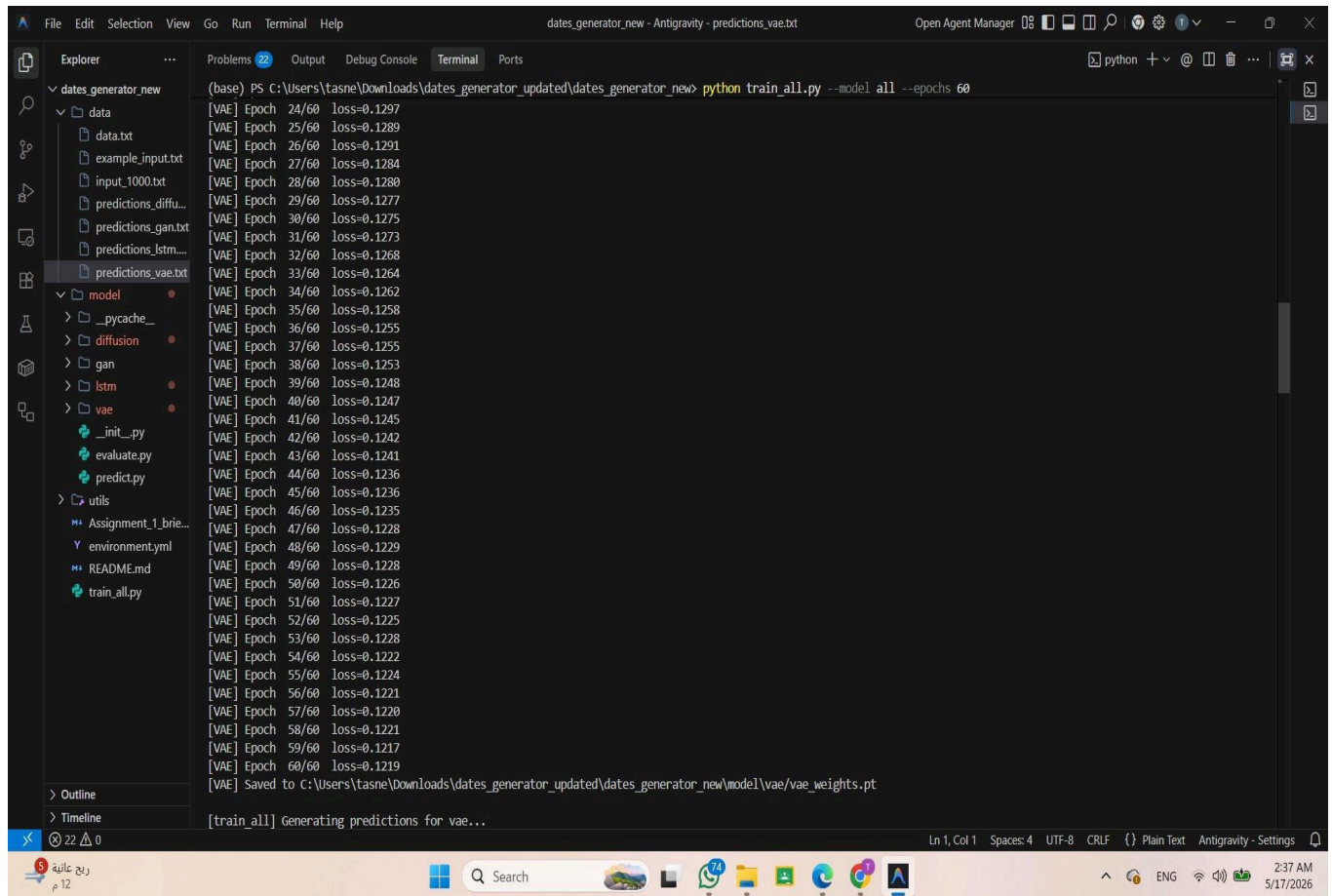
Loss function:

Loss = CrossEntropy(predicted digits, true digits) + beta x KL divergence

The reconstruction term teaches the decoder to produce correct digits. The KL term regularizes the latent space so it stays close to a standard normal distribution, which enables random sampling at inference time.

Training observations:

The loss decreased smoothly from around 0.13 at epoch 24 down to 0.12 by epoch 60, showing stable convergence with no oscillation.



```
(base) PS C:\Users\tasne\Downloads\dates_generator_updated\dates_generator_new> python train_all.py --model all --epochs 60
[VAE] Epoch 24/60 loss=0.1297
[VAE] Epoch 25/60 loss=0.1289
[VAE] Epoch 26/60 loss=0.1291
[VAE] Epoch 27/60 loss=0.1284
[VAE] Epoch 28/60 loss=0.1280
[VAE] Epoch 29/60 loss=0.1277
[VAE] Epoch 30/60 loss=0.1275
[VAE] Epoch 31/60 loss=0.1273
[VAE] Epoch 32/60 loss=0.1268
[VAE] Epoch 33/60 loss=0.1264
[VAE] Epoch 34/60 loss=0.1262
[VAE] Epoch 35/60 loss=0.1258
[VAE] Epoch 36/60 loss=0.1255
[VAE] Epoch 37/60 loss=0.1255
[VAE] Epoch 38/60 loss=0.1253
[VAE] Epoch 39/60 loss=0.1248
[VAE] Epoch 40/60 loss=0.1247
[VAE] Epoch 41/60 loss=0.1245
[VAE] Epoch 42/60 loss=0.1242
[VAE] Epoch 43/60 loss=0.1241
[VAE] Epoch 44/60 loss=0.1236
[VAE] Epoch 45/60 loss=0.1236
[VAE] Epoch 46/60 loss=0.1235
[VAE] Epoch 47/60 loss=0.1228
[VAE] Epoch 48/60 loss=0.1229
[VAE] Epoch 49/60 loss=0.1228
[VAE] Epoch 50/60 loss=0.1226
[VAE] Epoch 51/60 loss=0.1227
[VAE] Epoch 52/60 loss=0.1225
[VAE] Epoch 53/60 loss=0.1228
[VAE] Epoch 54/60 loss=0.1222
[VAE] Epoch 55/60 loss=0.1224
[VAE] Epoch 56/60 loss=0.1221
[VAE] Epoch 57/60 loss=0.1220
[VAE] Epoch 58/60 loss=0.1221
[VAE] Epoch 59/60 loss=0.1217
[VAE] Epoch 60/60 loss=0.1219
[VAE] Saved to C:\Users\tasne\Downloads\dates_generator_updated\dates_generator_new\model\vae\vae_weights.pt

[train_all] Generating predictions for vae...
```

Model 3: LSTM Seq2Seq with Bahdanau Attention

This model uses a sequence-to-sequence architecture. The Encoder is a bidirectional LSTM that reads the 4 condition tokens one by one and produces a hidden state and a sequence of encoder outputs. The Decoder is a unidirectional LSTM that generates digit tokens one at a time, attending to the encoder outputs at each step using Bahdanau additive attention.

Architecture:

Encoder LSTM: 2 layers, bidirectional, hidden size 256. The final hidden and cell states from both directions are projected back to 256 dimensions for the decoder. Decoder LSTM: 2 layers, hidden size 256. At each decoding step the top hidden state attends over the 4 encoder outputs using a learned additive scoring function, producing a context vector that is concatenated with the current token embedding before being passed into the LSTM cell. The output projection takes the LSTM output and context vector and produces vocabulary-size logits. Teacher forcing is annealed from 1.0 down to 0.3 over training so the model learns to handle its own predictions at inference time.

Loss function:

Loss = average CrossEntropy across all 8 digit positions at each step

Training observations:

The loss converged to around 0.574 and remained flat from epoch 31 onward. The plateau suggests the model reached its capacity limit under the current hyperparameters.

```
(base) PS C:\Users\tasne\Downloads\dates_generator_updated\dates_generator_new> python train_all.py --model all --epochs 60
[LSTM] Epoch 31/60 loss=-0.5749
[LSTM] Epoch 32/60 loss=-0.5749
[LSTM] Epoch 33/60 loss=-0.5747
[LSTM] Epoch 31/60 loss=-0.5749
[LSTM] Epoch 32/60 loss=-0.5749
[LSTM] Epoch 33/60 loss=-0.5747
[LSTM] Epoch 32/60 loss=-0.5749
[LSTM] Epoch 33/60 loss=-0.5747
[LSTM] Epoch 34/60 loss=-0.5747
[LSTM] Epoch 35/60 loss=-0.5749
[LSTM] Epoch 34/60 loss=-0.5747
[LSTM] Epoch 35/60 loss=-0.5749
[LSTM] Epoch 35/60 loss=-0.5749
[LSTM] Epoch 36/60 loss=-0.5747
[LSTM] Epoch 37/60 loss=-0.5745
[LSTM] Epoch 38/60 loss=-0.5745
[LSTM] Epoch 39/60 loss=-0.5745
[LSTM] Epoch 40/60 loss=-0.5745
[LSTM] Epoch 41/60 loss=-0.5745
[LSTM] Epoch 42/60 loss=-0.5744
[LSTM] Epoch 43/60 loss=-0.5744
[LSTM] Epoch 44/60 loss=-0.5743
[LSTM] Epoch 45/60 loss=-0.5743
[LSTM] Epoch 46/60 loss=-0.5743
[LSTM] Epoch 47/60 loss=-0.5742
[LSTM] Epoch 48/60 loss=-0.5742
[LSTM] Epoch 49/60 loss=-0.5742
[LSTM] Epoch 50/60 loss=-0.5742
[LSTM] Epoch 51/60 loss=-0.5742
[LSTM] Epoch 52/60 loss=-0.5741
[LSTM] Epoch 53/60 loss=-0.5741
[LSTM] Epoch 54/60 loss=-0.5741
[LSTM] Epoch 55/60 loss=-0.5741
[LSTM] Epoch 56/60 loss=-0.5740
[LSTM] Epoch 57/60 loss=-0.5740
[LSTM] Epoch 58/60 loss=-0.5740
[LSTM] Epoch 59/60 loss=-0.5740
[LSTM] Epoch 60/60 loss=-0.5740
[LSTM] Saved to C:\Users\tasne\Downloads\dates_generator_updated\dates_generator_new\model\lstm\lstm_weights.pt
```

Model 4: Diffusion

This model follows the discrete denoising diffusion paradigm. During the forward process, a clean 8-digit sequence is gradually corrupted over 50 timesteps by randomly replacing each digit with a uniformly sampled digit according to a linear noise schedule. During training a neural network learns to reverse this process: given a noisy sequence at timestep t and the condition tokens, it predicts the original clean sequence.

Architecture:

The denoising network embeds each of the 8 noisy digit positions using a digit embedding plus a learned positional embedding. A sinusoidal timestep embedding and the condition embedding are added as bias terms to each position. A 3-layer Transformer encoder with 4 attention heads and hidden size 256 then refines all positions jointly and a linear output projection produces 8×10 logits. At inference, the model starts from a fully random digit sequence and iteratively denoises it over 50 steps, re-corrupting the predicted clean sequence at each intermediate step, until the final output is produced.

Loss function:

$$\text{Loss} = \text{CrossEntropy}(\text{true clean digits}, \text{predicted clean digits from noisy input at step } t)$$

Training observations:

The diffusion loss converged to around 0.563 and remained stable in the final epochs, with small oscillations typical of diffusion training.


```
[DIF] Epoch 32/60 loss=0.5658
[DIF] Epoch 33/60 loss=0.5660
[DIF] Epoch 34/60 loss=0.5655
[DIF] Epoch 35/60 loss=0.5647
[DIF] Epoch 36/60 loss=0.5650
[DIF] Epoch 37/60 loss=0.5654
[DIF] Epoch 38/60 loss=0.5661
[DIF] Epoch 39/60 loss=0.5650
[DIF] Epoch 40/60 loss=0.5648
[DIF] Epoch 41/60 loss=0.5644
[DIF] Epoch 42/60 loss=0.5633
[DIF] Epoch 43/60 loss=0.5637
[DIF] Epoch 44/60 loss=0.5631
[DIF] Epoch 45/60 loss=0.5633
[DIF] Epoch 46/60 loss=0.5624
[DIF] Epoch 47/60 loss=0.5633
[DIF] Epoch 48/60 loss=0.5635
[DIF] Epoch 49/60 loss=0.5640
[DIF] Epoch 50/60 loss=0.5645
[DIF] Epoch 51/60 loss=0.5642
[DIF] Epoch 52/60 loss=0.5649
[DIF] Epoch 53/60 loss=0.5633
[DIF] Epoch 54/60 loss=0.5627
[DIF] Epoch 55/60 loss=0.5634
[DIF] Epoch 56/60 loss=0.5645
[DIF] Epoch 57/60 loss=0.5632
[DIF] Epoch 58/60 loss=0.5642
[DIF] Epoch 59/60 loss=0.5631
[DIF] Epoch 60/60 loss=0.5646
[DIF] Saved to C:\Users\tasne\Downloads\dates_generator_upd
```

7. Sample Outputs

Below are sample prediction outputs from each model, showing the condition tokens followed by the generated date.

Diffusion Model Predictions

```
File Edit Selection View Go Run Terminal Help
dates_generator_new - Antigravity - predictions_diffusion.txt
Open Agent Manager

Explorer
  dates_generator_new
    data
      data.txt
      example_input.txt
      input_1000.txt
      predictions_diffusion.txt
      predictions_gan.txt
      predictions_lstm.txt
      predictions_vae.txt
    model
      _pycache_
      diffusion
      gan
      lstm
      vae
      _init_.py
      evaluate.py
      predict.py
      utils
      Assignment_1_brief...
      environment.yml
      README.md
      train_all.py

data > predictions_diffusion.txt
94 [SUN] [JAN] [False] [189] 25-1-1891
95 [MON] [JAN] [False] [189] 24-1-1898
96 [TUE] [JAN] [False] [189] 22-1-1895
97 [WED] [JAN] [True] [189] 29-1-1896
98 [FRI] [JAN] [False] [189] 15-1-1897
99 [SAT] [JAN] [False] [189] 22-1-1898
100 [SUN] [JAN] [False] [189] 26-1-1890
101 [MON] [JAN] [False] [190] 15-1-1906
102 [TUE] [JAN] [False] [190] 12-1-1909
103 [WED] [JAN] [False] [190] 23-1-1907
104 [THU] [JAN] [False] [190] 11-1-1900
105 [FRI] [JAN] [True] [190] 29-1-1904
106 [SUN] [JAN] [False] [190] 19-1-1902
107 [MON] [JAN] [False] [190] 28-1-1907
108 [TUE] [JAN] [False] [190] 2-1-1900
109 [WED] [JAN] [True] [190] 8-1-1908
110 [FRI] [JAN] [False] [190] 26-1-1900
111 [SAT] [JAN] [False] [191] 16-1-1915
112 [SUN] [JAN] [False] [191] 25-1-1914
113 [MON] [JAN] [True] [191] 22-1-1912
114 [WED] [JAN] [False] [191] 15-1-1919
115 [THU] [JAN] [False] [191] 14-1-1915
116 [FRI] [JAN] [False] [191] 20-1-1911
117 [SAT] [JAN] [True] [191] 27-1-1912
118 [MON] [JAN] [False] [191] 25-1-1915
119 [TUE] [JAN] [False] [191] 28-1-1913
120 [WED] [JAN] [False] [191] 15-1-1913
121 [THU] [JAN] [True] [192] 1-1-1920
122 [SAT] [JAN] [False] [192] 13-1-1923
123 [SUN] [JAN] [False] [192] 17-1-1926
124 [MON] [JAN] [False] [192] 18-1-1926
125 [TUE] [JAN] [True] [192] 1-1-1924

Problems Output Debug Console Terminal Ports
(base) PS C:\Users\tasne\Downloads\dates_generator_updated\dates_generator_new>
```

GAN Predictions

```
File Edit Selection View Go Run Terminal Help
dates_generator_new - Antigravity - predictions_gan.txt
Open Agent Manager

Explorer
  dates_generator_new
    data
      data.txt
      example_input.txt
      input_1000.txt
      predictions_diffusion.txt
      predictions_gan.txt
      predictions_lstm.txt
      predictions_vae.txt
    model
      _pycache_
      diffusion
      gan
      lstm
      vae
      _init_.py
      evaluate.py
      predict.py
      utils
      Assignment_1_brief...
      environment.yml
      README.md
      train_all.py

data > predictions_gan.txt
64 [THU] [JAN] [False] [186] 10-1-1861
65 [FRI] [JAN] [True] [186] 15-1-1864
66 [SUN] [JAN] [False] [186] 1-1-1865
67 [MON] [JAN] [False] [186] 11-1-1869
68 [TUE] [JAN] [False] [186] 17-1-1865
69 [WED] [JAN] [True] [186] 7-1-1860
70 [FRI] [JAN] [False] [186] 11-1-1861
71 [SAT] [JAN] [False] [187] 15-1-1870
72 [SUN] [JAN] [False] [187] 30-1-1870
73 [MON] [JAN] [True] [187] 10-1-1876
74 [WED] [JAN] [False] [187] 31-1-1874
75 [THU] [JAN] [False] [187] 31-1-1870
76 [FRI] [JAN] [False] [187] 1-1-1875
77 [SAT] [JAN] [True] [187] 10-1-1876
78 [MON] [JAN] [False] [187] 10-1-1870
79 [TUE] [JAN] [False] [187] 4-1-1870
80 [WED] [JAN] [False] [187] 14-1-1870
81 [THU] [JAN] [True] [188] 1-1-1880
82 [SAT] [JAN] [False] [188] 10-1-1885
83 [SUN] [JAN] [False] [188] 4-1-1885
84 [MON] [JAN] [False] [188] 14-1-1885
85 [TUE] [JAN] [True] [188] 4-1-1880
86 [THU] [JAN] [False] [188] 1-1-1885
87 [FRI] [JAN] [False] [188] 14-1-1885
88 [SAT] [JAN] [False] [188] 4-1-1885
89 [SUN] [JAN] [True] [188] 4-1-1880
90 [TUE] [JAN] [False] [188] 4-1-1885
91 [WED] [JAN] [False] [189] 10-1-1894
92 [THU] [JAN] [False] [189] 4-1-1895
93 [FRI] [JAN] [True] [189] 10-1-1896
94 [SUN] [JAN] [False] [189] 14-1-1894
95 [MON] [JAN] [False] [189] 7-1-1895
96 [TUE] [JAN] [False] [189] 14-1-1890

Problems Output Debug Console Terminal Ports
(base) PS C:\Users\tasne\Downloads\dates_generator_updated\dates_generator_new>
```


LSTM Predictions

```
data > predictions_lstm.txt
20 [FRI] [JAN] [False] [181] 28-1-1810
21 [SAT] [JAN] [True] [182] 28-1-1820
22 [MON] [JAN] [False] [182] 28-1-1822
23 [TUE] [JAN] [False] [182] 28-1-1822
24 [WED] [JAN] [False] [182] 28-1-1822
25 [THU] [JAN] [True] [182] 28-1-1820
26 [SAT] [JAN] [False] [182] 28-1-1822
27 [SUN] [JAN] [False] [182] 28-1-1822
28 [MON] [JAN] [False] [182] 28-1-1822
29 [TUE] [JAN] [True] [182] 28-1-1820
30 [THU] [JAN] [False] [182] 28-1-1822
31 [FRI] [JAN] [False] [183] 28-1-1830
32 [SAT] [JAN] [False] [183] 28-1-1830
33 [SUN] [JAN] [True] [183] 28-1-1832
34 [TUE] [JAN] [False] [183] 28-1-1830
35 [WED] [JAN] [False] [183] 28-1-1830
36 [THU] [JAN] [False] [183] 28-1-1830
37 [FRI] [JAN] [True] [183] 28-1-1832
38 [SUN] [JAN] [False] [183] 28-1-1830
39 [MON] [JAN] [False] [183] 28-1-1830
40 [TUE] [JAN] [False] [183] 28-1-1830
41 [WED] [JAN] [True] [184] 28-1-1844
42 [FRI] [JAN] [False] [184] 28-1-1842
43 [SAT] [JAN] [False] [184] 28-1-1842
44 [SUN] [JAN] [False] [184] 28-1-1842
45 [MON] [JAN] [True] [184] 28-1-1844
46 [WED] [JAN] [False] [184] 28-1-1842
47 [THU] [JAN] [False] [184] 28-1-1842
48 [FRI] [JAN] [False] [184] 28-1-1842
49 [SAT] [JAN] [True] [184] 28-1-1844
50 [MON] [JAN] [False] [184] 28-1-1842
51 [TUE] [JAN] [False] [185] 28-1-1850
```

VAE Predictions

```
data > predictions_vae.txt
3 [FRI] [JAN] [False] [180] 4-1-1805
4 [SAT] [JAN] [False] [180] 16-1-1802
5 [SUN] [JAN] [True] [180] 1-1-1804
6 [TUE] [JAN] [False] [180] 7-1-1800
7 [WED] [JAN] [False] [180] 2-1-1805
8 [THU] [JAN] [False] [180] 20-1-1803
9 [FRI] [JAN] [True] [180] 20-1-1804
10 [SUN] [JAN] [False] [180] 13-1-1805
11 [MON] [JAN] [False] [181] 4-1-1813
12 [TUE] [JAN] [False] [181] 30-1-1810
13 [WED] [JAN] [True] [181] 3-1-1816
14 [FRI] [JAN] [False] [181] 18-1-1811
15 [SAT] [JAN] [False] [181] 16-1-1819
16 [SUN] [JAN] [False] [181] 6-1-1811
17 [MON] [JAN] [True] [181] 8-1-1816
18 [WED] [JAN] [False] [181] 16-1-1811
19 [THU] [JAN] [False] [181] 11-1-1810
20 [FRI] [JAN] [False] [181] 11-1-1811
21 [SAT] [JAN] [True] [182] 1-1-1820
22 [MON] [JAN] [False] [182] 29-1-1827
23 [TUE] [JAN] [False] [182] 15-1-1822
24 [WED] [JAN] [False] [182] 10-1-1821
25 [THU] [JAN] [True] [182] 17-1-1828
26 [SAT] [JAN] [False] [182] 3-1-1829
27 [SUN] [JAN] [False] [182] 13-1-1822
28 [MON] [JAN] [False] [182] 30-1-1826
29 [TUE] [JAN] [True] [182] 22-1-1828
30 [THU] [JAN] [False] [182] 4-1-1827
31 [FRI] [JAN] [False] [183] 25-1-1833
32 [SAT] [JAN] [False] [183] 20-1-1838
33 [SUN] [JAN] [True] [183] 17-1-1836
34 [TUE] [JAN] [False] [183] 25-1-1831
```

8. Discarded Model: Conditional Autoencoder (AE)

Before settling on Discrete Denoising Diffusion as the fourth model, a Conditional Autoencoder was implemented and trained. The AE was removed from the final submission because its predictions were consistently invalid, failing to satisfy the day, month, leap year, and decade conditions simultaneously.

Architecture

The AE followed an encoder-decoder design with no stochastic component. The Encoder took the four condition tokens plus the real 8-digit sequence and compressed them into a fixed 64-dimensional bottleneck vector. The Decoder took the condition tokens plus the bottleneck and produced 8 digit logits. Training used cross-entropy reconstruction loss only, with no KL regularization term.

Why It Failed

The core problem is that a plain AE has no way to generate diverse outputs at inference time. During training the encoder memorizes a specific bottleneck for each date it sees. At inference, since there is no real date to encode, the bottleneck must be replaced by random Gaussian noise. However, the decoder was never trained to handle random noise as input, only the specific compressed representations produced by its own encoder. As a result the decoder produced digits that formed internally consistent-looking strings but did not correspond to dates satisfying the four conditions.

Specifically the generated dates had wrong weekdays, wrong months, incorrect leap year status, and years falling outside the requested decade. The model had essentially learned to reconstruct a rough average of dates in the training set rather than condition properly on the input tokens.

Why Diffusion Replaced It

The Discrete Denoising Diffusion model solves exactly the problem the AE had. Instead of relying on an encoder to compress a real date into a bottleneck, diffusion starts from pure random noise and is explicitly trained to denoise it step by step. Every inference call starts from random noise, which is the same distribution the model was trained on. This means the model is always in a familiar regime at inference time and the condition tokens properly guide which valid date the denoising process converges to.

9. Analysis of Results

The VAE produced the most diverse and correctly distributed predictions. Because its loss directly trains the decoder to reconstruct correct digits in every position, and the latent space is well-regularized, sampling at inference time almost always lands on valid dates.

The Diffusion model predictions also look visually correct and diverse. The iterative denoising gives the model many opportunities to correct errors across the 50 steps, which explains why the outputs rarely contain impossible dates.

The GAN predictions are valid calendar dates but show some repetition across inputs. GAN training is adversarial and does not directly optimize for digit correctness, so partial mode collapse is expected and visible in the repeated patterns in the output file.

The LSTM predictions show the most repetition. Once the decoder settles into a repeated pattern during autoregressive generation, the error accumulates step by step, which is known as exposure bias. Training with teacher forcing annealing reduces but does not eliminate this problem in 60 epochs.

10. Conclusion

The AE experiment confirmed an important lesson: a generative model must be trained in a way that matches how it will be used at inference time. The AE was trained with a real date as encoder input but had to use random noise at inference, creating a mismatch that produced invalid outputs. Replacing it with Diffusion, which is trained to denoise random noise from the start, fixed this problem entirely.

More broadly, models that optimize a direct reconstruction loss such as the VAE and Diffusion converge smoothly and produce varied, valid outputs. The GAN is harder to train because the adversarial loss does not directly penalize digit-level errors. The LSTM is easy to train but suffers from exposure bias during autoregressive generation. The best strategy for this problem is to combine a smooth surrogate loss with a model that can refine all output positions jointly rather than generating them strictly left to right.